MCT344
Industrial Robotics

Faculty of Engineering - Ain Shams University
ICHEP
International Credit Hours Engineering Programs

AIN SHAMS UNIVERSITY

FACULTY OF ENGINEERING

International Credit Hours Engineering Programs (I.CHEP)

MCT344

Industrial Robotics

Second Milestone

Supervised By:

Dr.Shady Ahmed,

Eng.Mina Yousry,

Eng. Dina Zakaria.

Team Members

Name	ID
Philopateer Elkes	21P0240
Yahia Mohamed	21P0041

Table of Contents

Abstract: 5
1.0 Introduction 6
2.0CAD Model Import & URDF Conversion 7
3.0 Configuring ROS2 Controllers 9
4.0 Creating the Launch File 11
5.0. MOVEIT SETUP ASSISTANT 14
5.1 Collison Matrix 14
5.2 Planning Groups 15
5.3 Robot Poses 15
5.4 ROS2 Controllers 19
5.5 Moveit Controllers 19
6.0JOINT Configuration and Limits 20
7.0Moveit Controllers 22
8.0 LAUNCH FILE 23
9.0 Simulation 24
10.0 Drive Link 25
11.0 Repo Link 25

List of figures:

Figure 1:CAD Model 7
Figure 2:Joint Names 8
Figure 3:Joint controller.yaml 9
Figure 4:Joint controller.yaml 2 10
Figure 5:Launch file 1 11
Figure 6:Launch file 2 12
Figure 7:Launch file 3 13
Figure 8:Robot in Gazebo 13
Figure 9:Collison Matrix 14
Figure 10:Planning Groups 15
Figure 11:Zero Pose 15
Figure 12:Pos 1 16
Figure 13:Pos2 17
Figure 14:Gripper Fully Closed 18
Figure 15: Gripper Fully opened 18
Figure 16:ROS2 Controllers 19
Figure 17:5 Moveit Controllers 19
Figure 18: joint limits.\ 21
Figure 19:moveit_controllers.yaml 22
Figure 20:Launch File 23
Figure 21:Pose 0 24
Figure 22:Pose 1 24
Figure 23:Close Gripper 25

Abstract:

This report presents the implementation of a complete motion planning and execution pipeline for a 6-degree-of-freedom (6-DOF) robotic manipulator using MoveIt 2 within the ROS 2 framework. The objective is to simulate and control the robot's motion by integrating key components such as URDF-based robot modeling, trajectory planning, and visualization through RViz and Gazebo. A customized configuration package is generated using the MoveIt Setup Assistant and adapted to ensure seamless operation across the simulation and planning environments. To validate the system, the robot is tasked with reaching a series of goal positions derived from the team members' student ID digits, introducing project-specific variations. Final outputs include a working simulation, visual documentation, a detailed report, and a demonstration video highlighting the robot’s ability to perform accurate and reliable trajectory execution.

1.0 Introduction

The primary objective of this project is to showcase the implementation of motion planning and execution for a six-degree-of-freedom (6-DOF) robotic manipulator using the MoveIt 2 motion planning framework within the Robot Operating System 2 (ROS 2) ecosystem. This milestone extends core concepts in robotic simulation and control by constructing a complete and functional motion planning pipeline. The pipeline encompasses essential components such as robot modeling, trajectory generation, planning algorithm integration, and real-time visualization.

To initiate the setup, the MoveIt Setup Assistant is utilized to create a configuration package specifically adapted to the robot's Unified Robot Description Format (URDF) model. This package includes necessary launch files, planning configurations, and controllers. Subsequent customization and verification ensure seamless compatibility with both RViz—used for visualizing motion plans and joint configurations—and Gazebo, which provides realistic, physics-based simulation of the robot's dynamics and environment interactions.

For performance evaluation, the robotic manipulator is programmed to move to a set of predefined target poses. These target positions are uniquely derived from the last two digits of each team member's student ID, adding a layer of uniqueness and individualization to each team's implementation. This also tests the robot's ability to handle arbitrary goal states while maintaining accurate inverse kinematics and trajectory planning.

2.0CAD Model Import & URDF Conversion

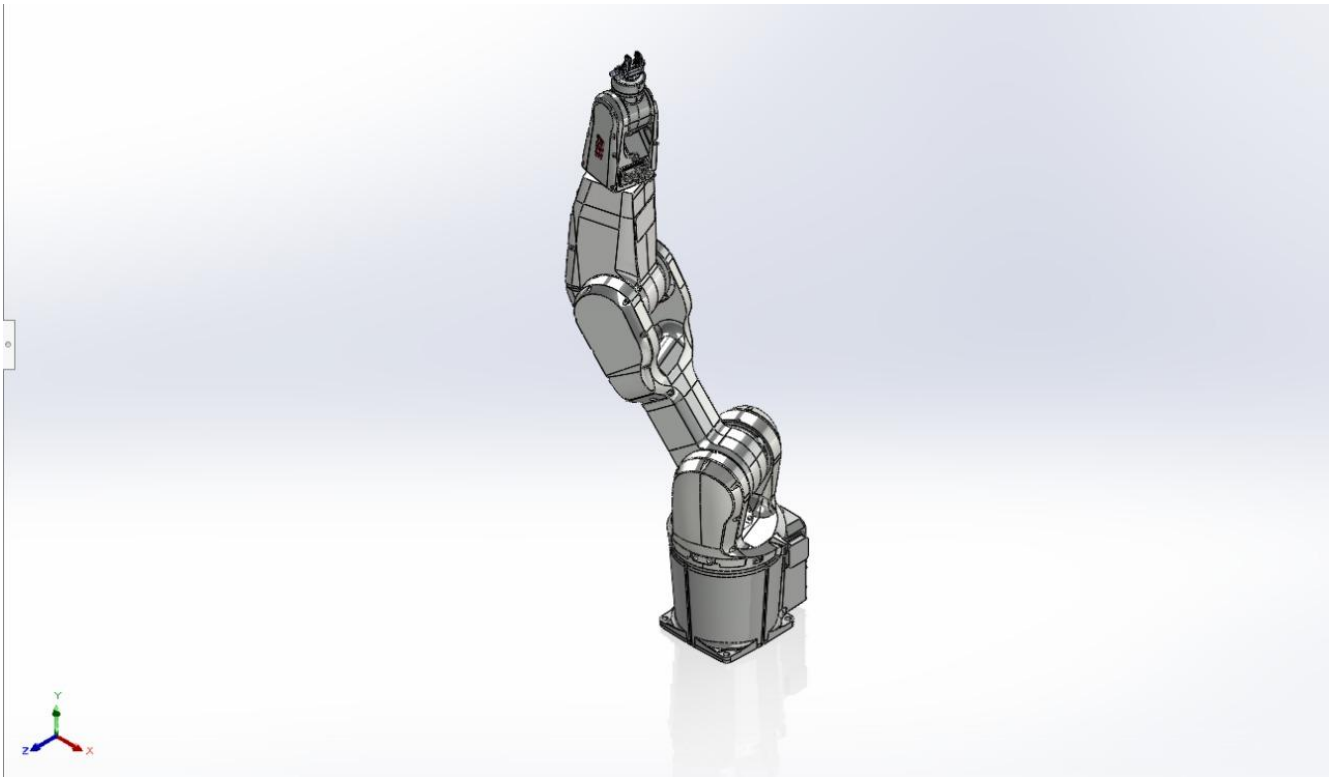

Figure 1:CAD Model

To integrate the robotic arm with ROS2, we exported the SolidWorks assembly as a URDF file using the SolidWorks URDF Exporter Plugin. During the export process, each link and joint was carefully named and assigned realistic mass and inertia values.

```xml
</hardware>
<joint name="joint1_21P0041_21P0240">
  <command_interface name="position" />
  <state_interface name="position">
    <param name="initial_value">0.0</param>
  </state_interface>
  <state_interface name="velocity">
    <param name="initial_value">0.0</param>
  </state_interface>
</joint>

<joint name="joint2_21P0041_21P0240">
  <command_interface name="position" />
  <state_interface name="position">
    <param name="initial_value">0.0</param>
  </state_interface>
  <state_interface name="velocity">
    <param name="initial_value">0.0</param>
  </state_interface>
</joint>

<joint name="joint3_21P0041_21P0240">
  <command_interface name="position" />
  <state_interface name="position">
    <param name="initial_value">0.0</param>
  </state_interface>
  <state_interface name="velocity">
    <param name="initial_value">0.0</param>
  </state_interface>
</joint>

<joint name="joint4_21P0041_21P0240">
  <command_interface name="position" />
  <state_interface name="position">
    <param name="initial_value">0.0</param>
  </state_interface>
  <state_interface name="velocity">
    <param name="initial_value">0.0</param>
  </state_interface>
</joint>

<joint name="joint5_21P0041_21P0240">
  <command_interface name="position" />
  <state_interface name="position">
```

Figure 2:Joint Names

3.0 Configuring ROS2 Controllers

```yaml
controller_manager:
  ros__parameters:
    update_rate: 100 # Hz
    arm_group_controller:
      type: joint_trajectory_controller/JointTrajectoryController
    hand_group_controller:
      type: joint_trajectory_controller/JointTrajectoryController
    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

arm_group_controller:
  ros__parameters:
    joints:
      - joint1_21P0041_21P0240
      - joint2_21P0041_21P0240
      - joint3_21P0041_21P0240
      - joint4_21P0041_21P0240
    interface_name: position
    command_interfaces:
      - position
    state_interfaces:
      - position
      - velocity
    allow_nonzero_velocity_at_trajectory_end: true
    action_monitor_rate: 20.0
    allow_partial_joints_goal: false
    open_loop_control: true
    constraints:
      stopped_velocity_tolerance: 0.01
      goal_time: 0.0
      joint1_21P0041_21P0240:
        trajectory: 0.05
        goal: 0.03
      joint2_21P0041_21P0240:
        trajectory: 0.05
        goal: 0.03
      joint3_21P0041_21P0240:
        trajectory: 0.05
        goal: 0.03
      joint4_21P0041_21P0240:
        trajectory: 0.05
        goal: 0.03

hand_group_controller:
  ros__parameters:
    joints:
      - joint5_21P0041_21P0240
      - joint6_21P0041_21P0240
```

Figure 3:Joint controller.yaml

```yaml

hand_group_controller:
  ros__parameters:
    joints:
      - joint5_21P0041_21P0240
      - joint6_21P0041_21P0240
      - joint7_21P0041_21P0240
      - joint8_21P0041_21P0240
    interface_name: position
    command_interfaces:
      - position
    state_interfaces:
      - position
      - velocity
    allow_nonzero_velocity_at_trajectory_end: true
    gains:
      joint5_21P0041_21P0240: { p: 500.0, i: 10.0, d: 10.0 }
      joint6_21P0041_21P0240: { p: 500.0, i: 10.0, d: 10.0 }

```

Figure 4:Joint controller.yaml 2

4.0 Creating the Launch File

```python
from launch import LaunchDescription
from launch.actions import (
    DeclareLaunchArgument,
    IncludeLaunchDescription,
    SetEnvironmentVariable,
)
from launch.substitutions import PathJoinSubstitution
from launch_ros.actions import Node
from launch.launch_description_sources import PythonLaunchDescriptionSource
from ament_index_python.packages import get_package_share_directory
import xacro
import os


def generate_launch_description():
    pkg_ros_gz_sim = get_package_share_directory("ros_gz_sim")
    pkg_lab_gazebo = get_package_share_directory("abb_robot")

    # Process the Xacro file to generate URDF
    xacro_file = os.path.join(pkg_lab_gazebo, "urdf", "robot_pkg.urdf")
    doc = xacro.parse(open(xacro_file))
    robot_description = doc.toxml()

    gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(pkg_ros_gz_sim, "launch", "gz_sim.launch.py")
        ),
        launch_arguments={"gz_args": "-r empty.sdf"}.items(),
    )

    # Robot State Publisher node
    robot_state_publisher_node = Node(
        package="robot_state_publisher",
        executable="robot_state_publisher",
        parameters=[{"robot_description": robot_description}],
        output="screen",
    )

    # Joint State Publisher node
    joint_state_publisher_node = Node(
        package="joint_state_publisher",
        executable="joint_state_publisher",
        name="joint_state_publisher",
        output="screen",
    )

    # Add the bridge node for mapping the topics between ROS and GZ sim
```

Figure 5:Launch file 1

```python
        package="ros_gz_bridge",
        executable="parameter_bridge",
        arguments=[
            # Clock (IGN -> ROS2)
            "/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock",
            # Joint states (IGN -> ROS2)
            "/world/empty/model/abb_robot/joint_state@sensor_msgs/msg/JointState[gz.msgs.JointState",
        ],
        remappings=[
            ("/world/empty/model/abb_robot/joint_state", "joint_states"),
        ],
        output="screen",
    )

    # Spawn Entity node
    spawn_entity = Node(
        package="ros_gz_sim",
        executable="create",
        arguments=["-name", "abb_robot", "-topic", "/robot_description"],
        parameters=[{"use_sim_time": True}],
        output="screen",
    )

    # add the robot controllers for joint trajectory, arm group and hand group controllers
    robot_controller = PathJoinSubstitution( [
        pkg_lab_gazebo,
        "config",
        "joint_controller.yaml", ]
    )

    # pass the controllers to the control node to send commands to the controllers
    control_node = Node(
        package="controller_manager",
        executable="ros2_control_node",
        parameters=[robot_controller],
        output="screen",
    )

    # Controller Manager Spawner node to launch the joint state broadcaster
    joint_state_broadcaster_spawner = Node(
        package="controller_manager",
        executable="spawner",
        arguments=["joint_state_broadcaster", "--param-file", robot_controller],
    )
    # Controller Manager Spawner node to launch the arm group controller
    arm_controller_manager_spawner = Node(
```

Figure 6:Launch file 2

```python
        arguments=["joint_state_broadcaster", "--param-file", robot_controller],
    )
    # Controller Manager Spawner node to launch the arm group controller
    arm_controller_manager_spawner = Node(
        package='controller_manager',
        executable='spawner',
        arguments=['arm_group_controller',"--param-file", robot_controller], output='screen'
    )
    # Controller Manager Spawner node to launch the hand group controller
    hand_controller_manager_spawner = Node(
        package='controller_manager',
        executable='spawner',
        arguments=['hand_group_controller', "--param-file", robot_controller], output='screen'
    )

    # Get the meshes path in the install directory
    meshes_path = os.path.join(pkg_lab_gazebo, "meshes")
    return LaunchDescription(
        [
            SetEnvironmentVariable("GZ_SIM_RESOURCE_PATH", meshes_path),
            gazebo,
            spawn_entity,
            bridge,
            robot_state_publisher_node,
            joint_state_publisher_node,
            control_node,
            arm_controller_manager_spawner,
            hand_controller_manager_spawner,
            joint_state_broadcaster_spawner,
        ]
    )

```

Figure 7:Launch file 3

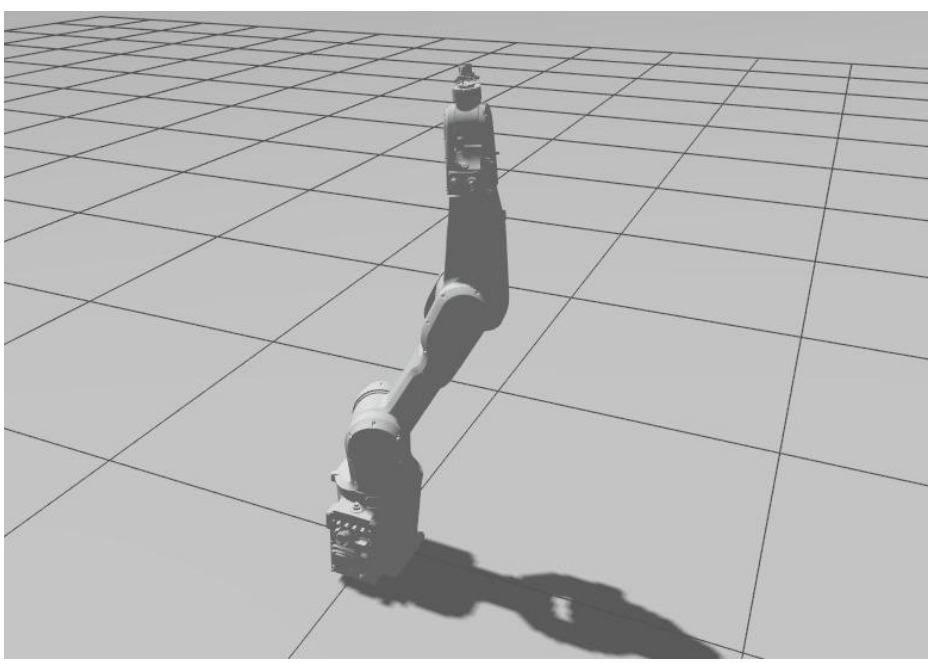

Figure 8:Robot in Gazebo

5.0. MOVEIT SETUP ASSISTANT

5.1 Collison Matrix

In this step, a collision matrix was configured to define the self-collision behavior between individual robot links.

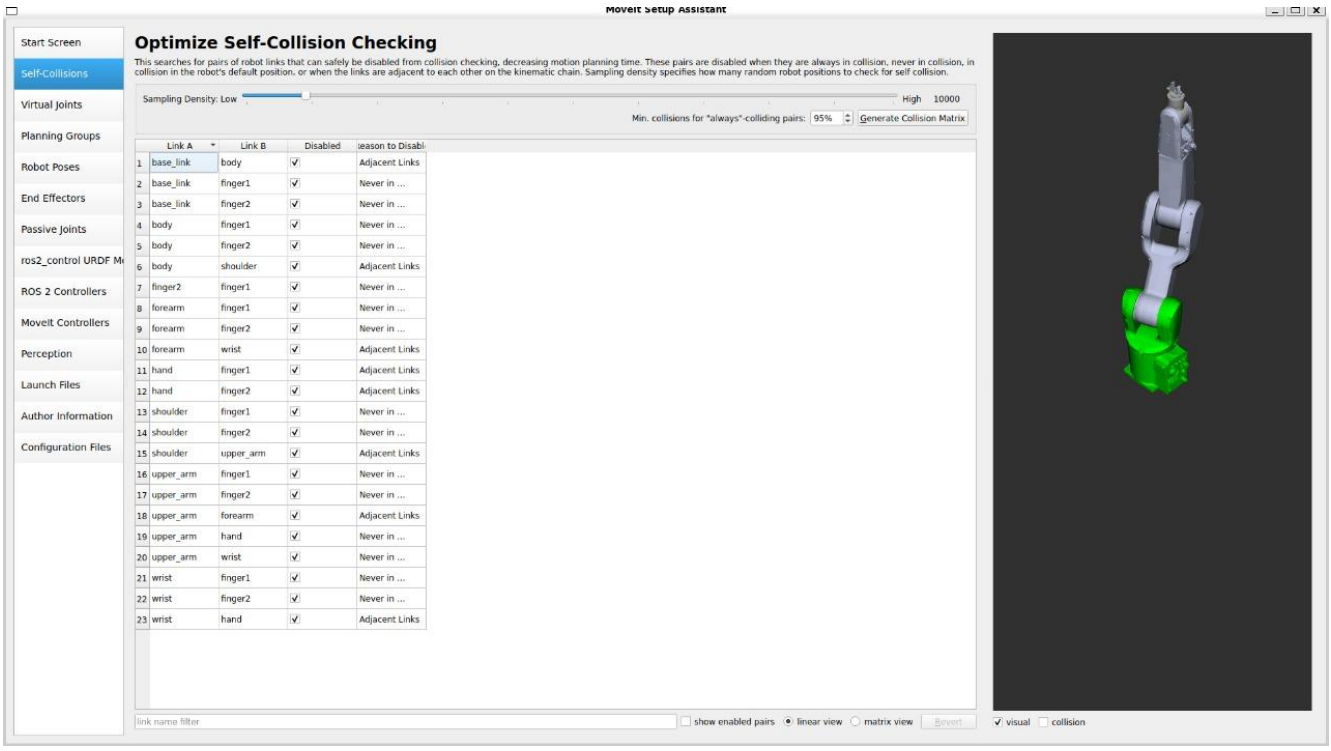

Figure 9:Collison Matrix

5.2 Planning Groups

Next, two planning groups were defined: one for the robotic arm, encompassing joints 1 through 6, and another for the hand (gripper), which includes joints 7 and 8.

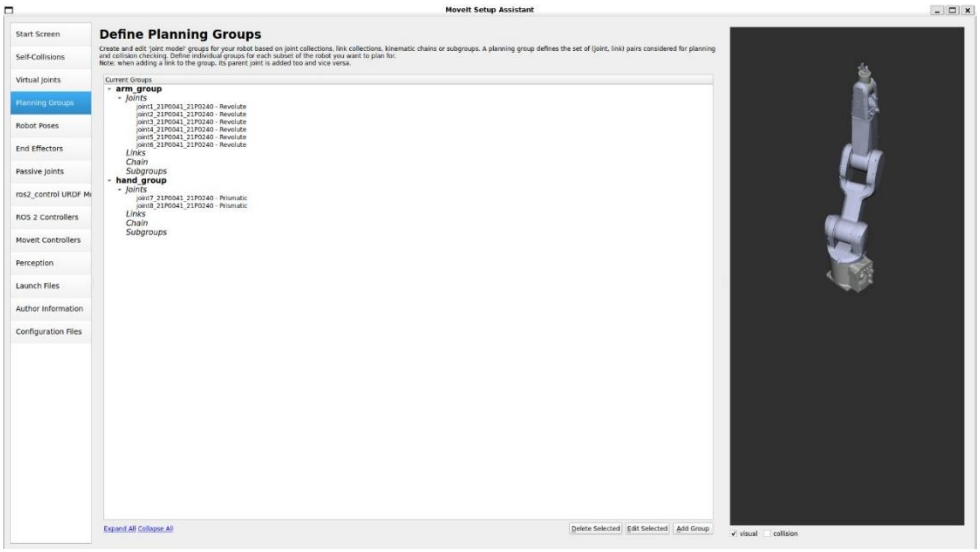

Figure 10:Planning Groups

5.3 Robot Poses

In this section, five distinct poses were configured for the robot—three assigned to the arm group and two to the hand (gripper) group.

- Pose 0: This pose is defined as the initial position of the robot, so all joints are zero degrees (0 rad).

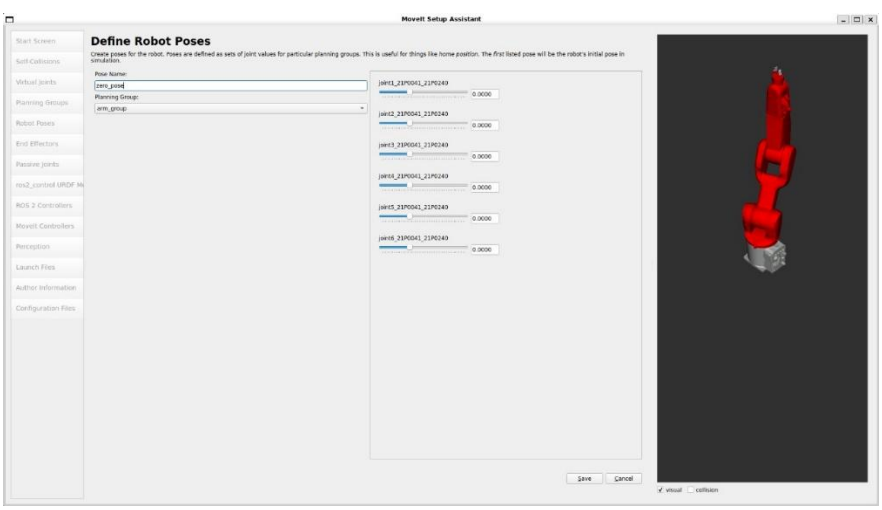

Figure 11:Zero Pose

<u>Pose 1</u>: In this pose the joints are defined with student id 21P0240

- **Joint 1**: $0 \times 10 = \mathbf{0°}$
- **Joint 2**: $4 \times 10 = \mathbf{40°}$
- **Joint 3**: $0 \times 5 = \mathbf{0°}$
- **Joint 4**: $4 \times 5 = \mathbf{20°}$
- **Joint 5**: $(0 + 2) \times 10 = \mathbf{20°}$
- **Joint 6**: $\mathbf{0°}$

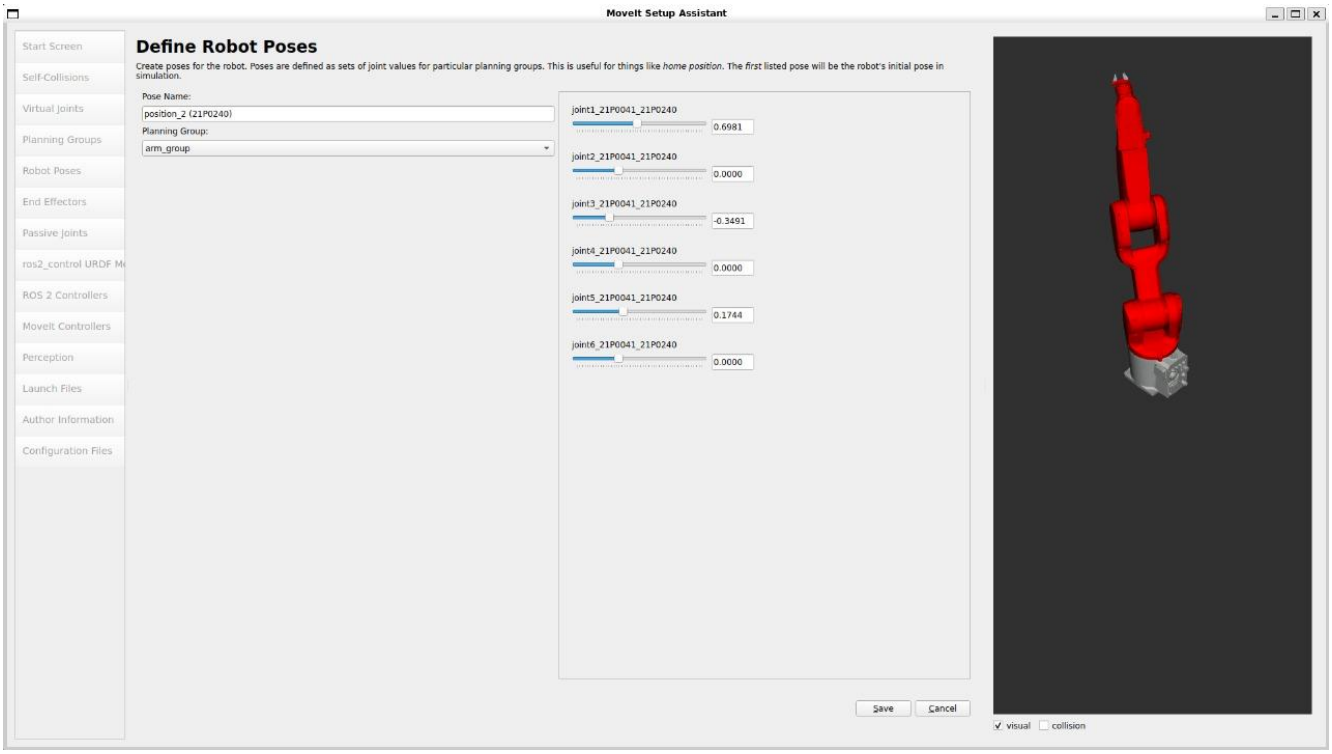

Figure 12:Pos 1

<u>Pose 2</u>: In this pose the joints are defined with student id 21P0041

- **joint 1**: $4 \times 10 =$ **40°**
- **Joint 2**: $1 \times 15 =$ **15°**
- **Joint 3**: $4 \times$ -5 = **-20°**
- **Joint 4**: $1 \times 5 =$ **5°**
- **Joint 5**: $(1 + 1) \times 10 =$ **20°**
- **Joint 6**: **0°**

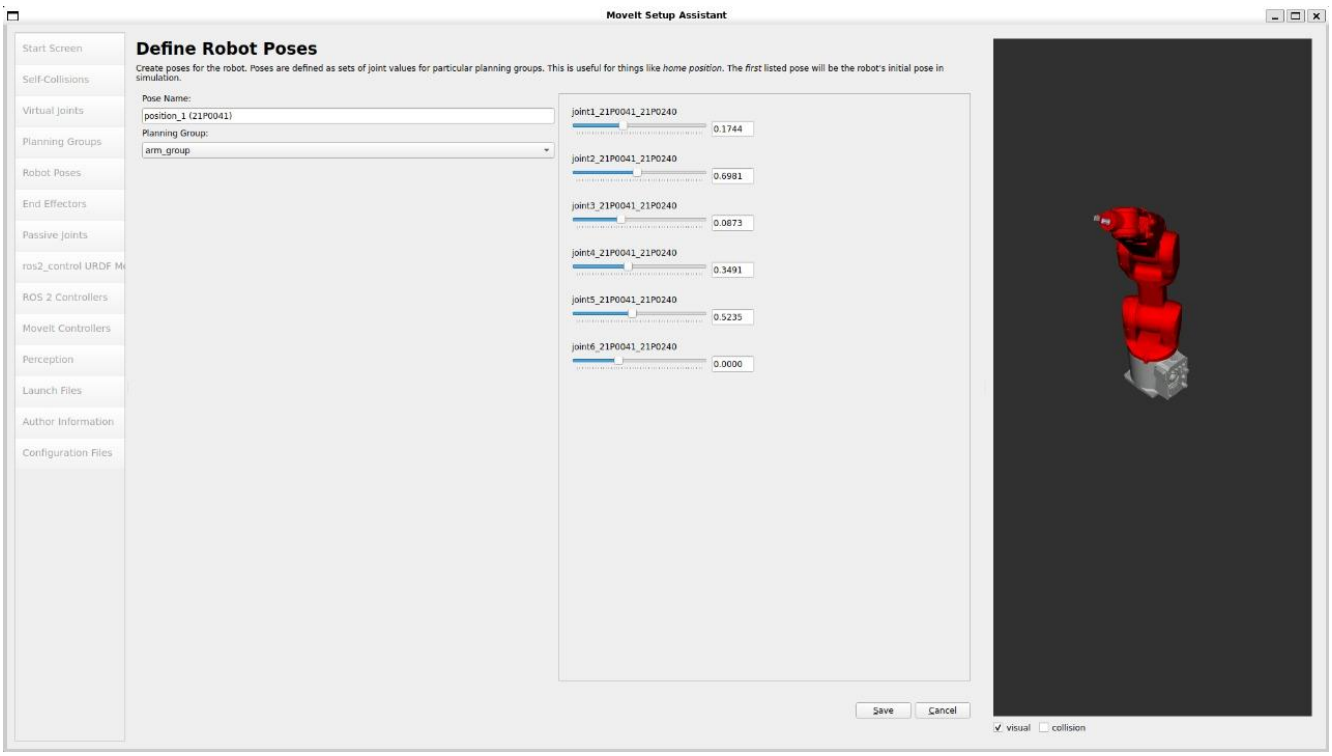

Figure 13:Pos2

- Close Hand pose: Gripper Fully closed

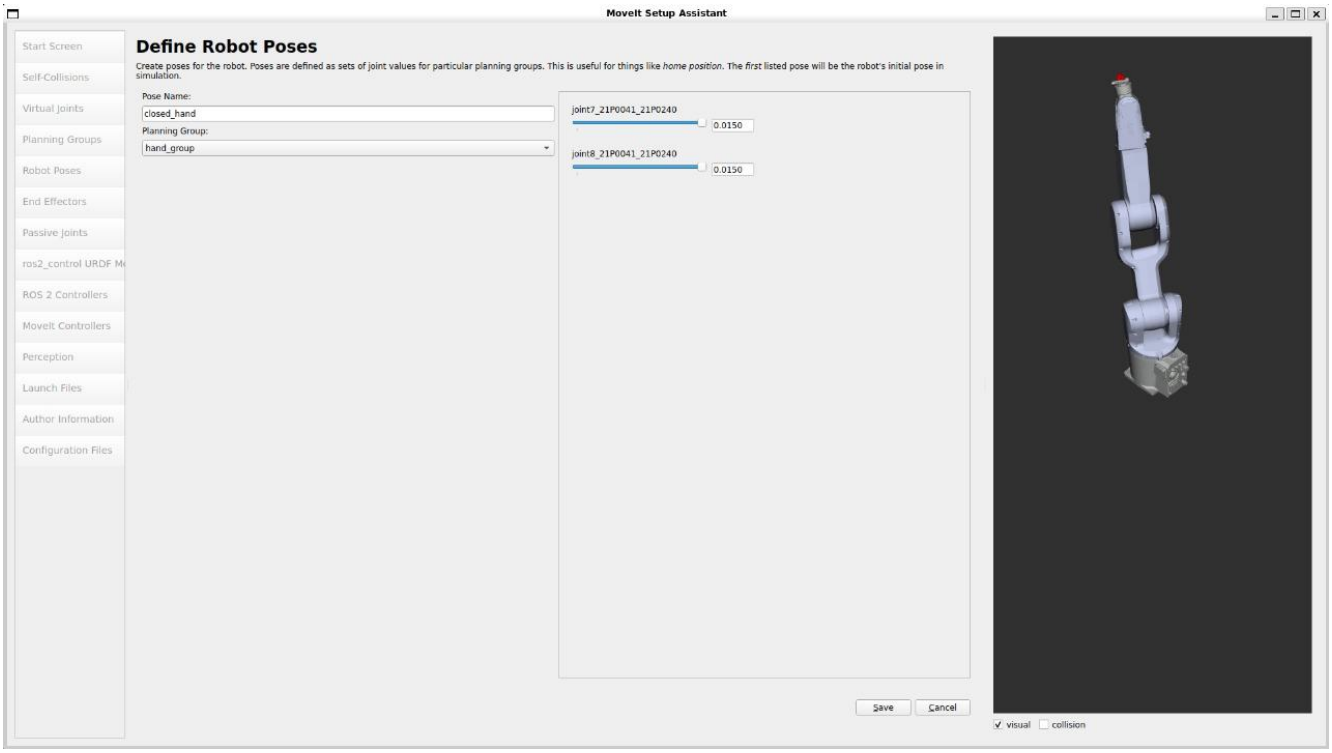

Figure 14:Gripper Fully Closed

- Open Hand Pose: Gripper Fully opened

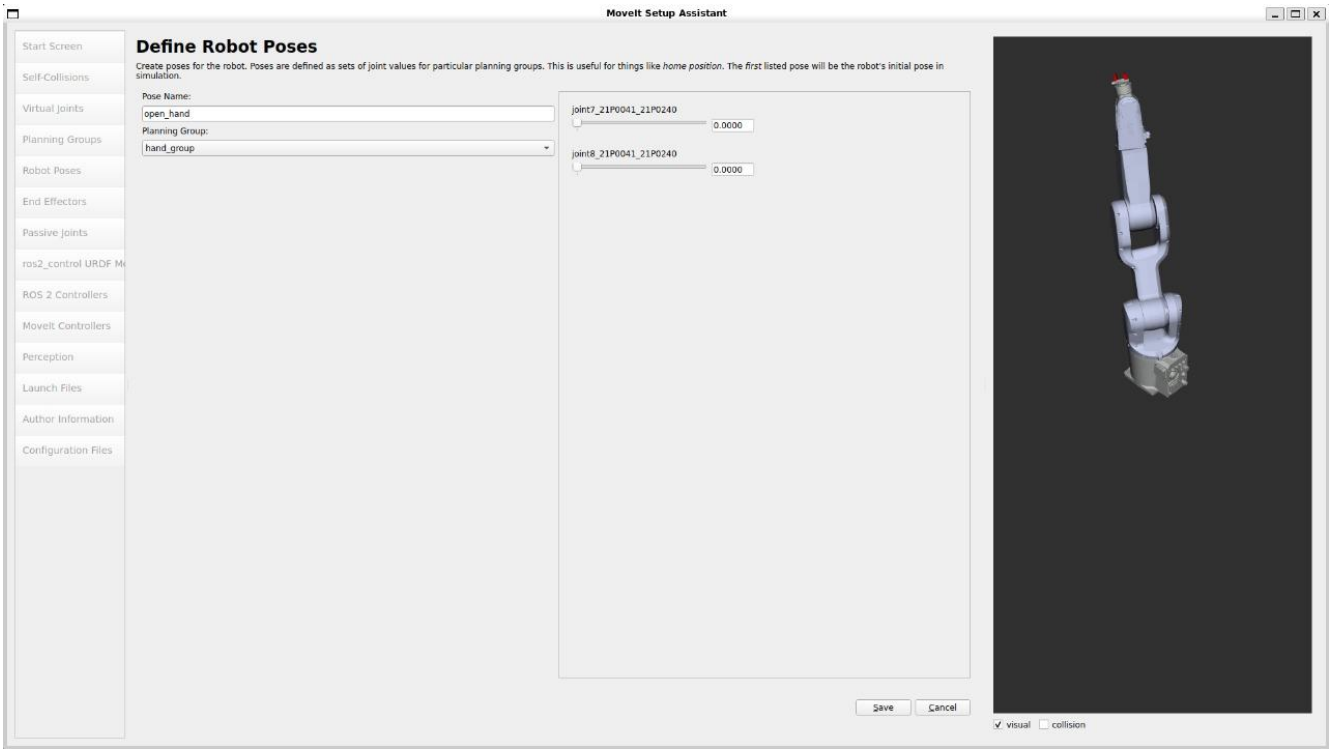

Figure 15: Gripper Fully opened

5.4 ROS2 Controllers

In this step, we configured two ROS 2 controllers—one dedicated to the arm group and the other to the hand (gripper) group.

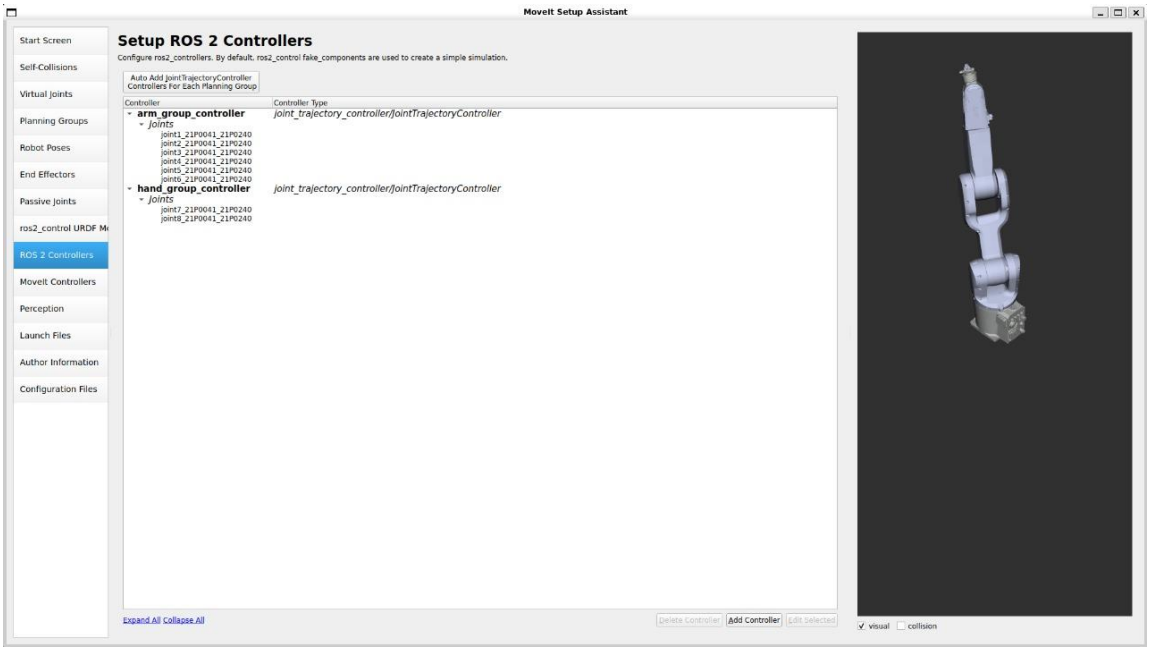

Figure 16:ROS2 Controllers

5.5 Moveit Controllers

Two MoveIt controllers were set up, each corresponding to one of the planning groups.

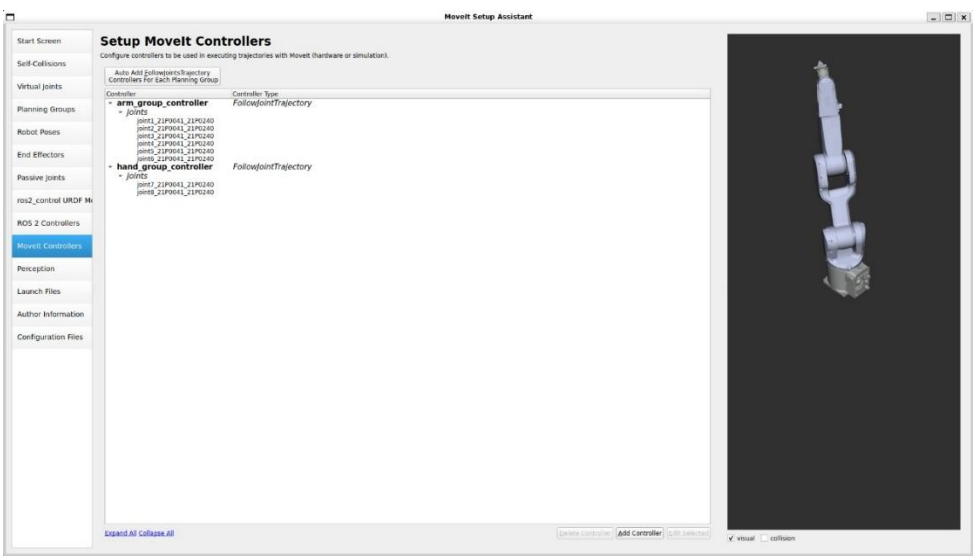

Figure 17:5 Moveit Controllers

6.0JOINT Configuration and Limits

The following are the initial joint positions.

joint_limits.yaml | initial_positions.yaml | robot_motion_planning.launch.py 6

D: > college > Year 4 senior 1 > semseter 2 > Industrial Robotics > project > Industrial-Robotics-Project > Ros Packages > abb_moveit > confi

```yaml
# Default initial positions for abb_robot's ros2_control fake system

initial_positions:
  joint1_21P0041_21P0240: 0.0
  joint2_21P0041_21P0240: 0.0
  joint3_21P0041_21P0240: 0.0
  joint4_21P0041_21P0240: 0.0
  joint5_21P0041_21P0240: 0.0
  joint6_21P0041_21P0240: 0.0
  joint7_21P0041_21P0240: 0.0
  joint8_21P0041_21P0240: 0.0
```

Here are joint limits.

File Edit Selection View Go Run Terminal Help

joint_limits.yaml

D: > college > Year 4 senior 1 > semseter 2 > Industrial Robotics > project > Industrial-Robotics-Project > Ros Packages > abb_moveit > config > joint_limits.yaml

```yaml
# joint_limits.yaml allows the dynamics properties specified in the URDF to be overwritten or augmented as needed

# For beginners, we downscale velocity and acceleration limits.
# You can always specify higher scaling factors (<= 1.0) in your motion requests.  # Increase the values below to 1.0 to always move at maximum speed.
default_velocity_scaling_factor: 0.1
default_acceleration_scaling_factor: 0.1

# Specific joint properties can be changed with the keys [max_position, min_position, max_velocity, max_acceleration]
# Joint limits can be turned off with [has_velocity_limits, has_acceleration_limits]
joint_limits:
  joint1_21P0041_21P0240:
    has_velocity_limits: true
    max_velocity: 3.0
    has_acceleration_limits: true
    max_acceleration: 5.0
  joint2_21P0041_21P0240:
    has_velocity_limits: true
    max_velocity: 3.0
    has_acceleration_limits: true
    max_acceleration: 5.0
  joint3_21P0041_21P0240:
    has_velocity_limits: true
    max_velocity: 3.0
    has_acceleration_limits: true
    max_acceleration: 5.0
  joint4_21P0041_21P0240:
    has_velocity_limits: true
    max_velocity: 3.0
    has_acceleration_limits: true
    max_acceleration: 5.0
  joint5_21P0041_21P0240:
    has_velocity_limits: true
    max_velocity: 3.0
    has_acceleration_limits: true
    max_acceleration: 5.0
  joint6_21P0041_21P0240:
    has_velocity_limits: true
    max_velocity: 3.0
    has_acceleration_limits: true
    max_acceleration: 5.0
  joint7_21P0041_21P0240:
    has_velocity_limits: true
    max_velocity: 3.0
    has_acceleration_limits: true
    max_acceleration: 5.0
  joint8_21P0041_21P0240:
```

```yaml
  joint4_21P0041_21P0240:
    has_velocity_limits: true
    max_velocity: 3.0
    has_acceleration_limits: true
    max_acceleration: 5.0
  joint5_21P0041_21P0240:
    has_velocity_limits: true
    max_velocity: 3.0
    has_acceleration_limits: true
    max_acceleration: 5.0
  joint6_21P0041_21P0240:
    has_velocity_limits: true
    max_velocity: 3.0
    has_acceleration_limits: true
    max_acceleration: 5.0
  joint7_21P0041_21P0240:
    has_velocity_limits: true
    max_velocity: 3.0
    has_acceleration_limits: true
    max_acceleration: 5.0
  joint8_21P0041_21P0240:
    has_velocity_limits: true
    max_velocity: 3.0
    has_acceleration_limits: true
    max_acceleration: 5.0
```

Figure 18: joint limits.

7.0Moveit Controllers

Here is moveit_controllers.yaml file.

joint_limits.yaml | moveit_controllers.yaml

D: > college > Year 4 senior 1 > semseter 2 > Industrial Robotics > project > Industrial-Robotics-Project > Ros Packages > abb_moveit > config > move

```yaml
# MoveIt uses this configuration for controller management

moveit_controller_manager: moveit_simple_controller_manager/MoveItSimpleControllerManager

moveit_simple_controller_manager:
  controller_names:
    - arm_group_controller
    - hand_group_controller

  arm_group_controller:
    type: FollowJointTrajectory
    action_ns: follow_joint_trajectory
    default: true
    joints:
      - joint1_21P0041_21P0240
      - joint2_21P0041_21P0240
      - joint3_21P0041_21P0240
      - joint4_21P0041_21P0240
      - joint5_21P0041_21P0240
      - joint6_21P0041_21P0240
  hand_group_controller:
    type: FollowJointTrajectory
    action_ns: follow_joint_trajectory
    default: true
    joints:
      - joint7_21P0041_21P0240
      - joint8_21P0041_21P0240
```

Figure 19:moveit_controllers.yaml

8.0 LAUNCH FILE

Here is our launch file that starts and connects both the gz-sim and RViz nodes.

moveit_gazebo.launch.py

D: > college > Year 4 senior 1 > semseter 2 > Industrial Robotics > project > Industrial-Robotics-Project > Ros Packages > abb_moveit > launch > moveit_gazebo.launch.py > ...

```python
from launch import LaunchDescription
from launch_ros.actions import SetParameter
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
import os
from launch.actions import TimerAction
from ament_index_python.packages import get_package_share_directory


def generate_launch_description():
    # Launch Gazebo from abb_robot package
    gazebo_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(
                get_package_share_directory("abb_robot"), "launch/gazebo.launch.py"
            )
        )
    )

    # Launch MoveIt demo from abb_moveit package
    moveit_launch = TimerAction(
        period=5.0,  # Delay in seconds
        actions=[
            IncludeLaunchDescription(
                PythonLaunchDescriptionSource(
                    os.path.join(
                        get_package_share_directory("abb_moveit"),
                        "launch/demo.launch.py",
                    )
                )
            )
        ],
    )

    return LaunchDescription(
        [
            # Use simulation time
            SetParameter(name="use_sim_time", value=True),
            # Include both launch files
            gazebo_launch,
            moveit_launch,
        ]
    )

```

Figure 20:Launch File

9.0 Simulation

Here is a simulation for All poses.

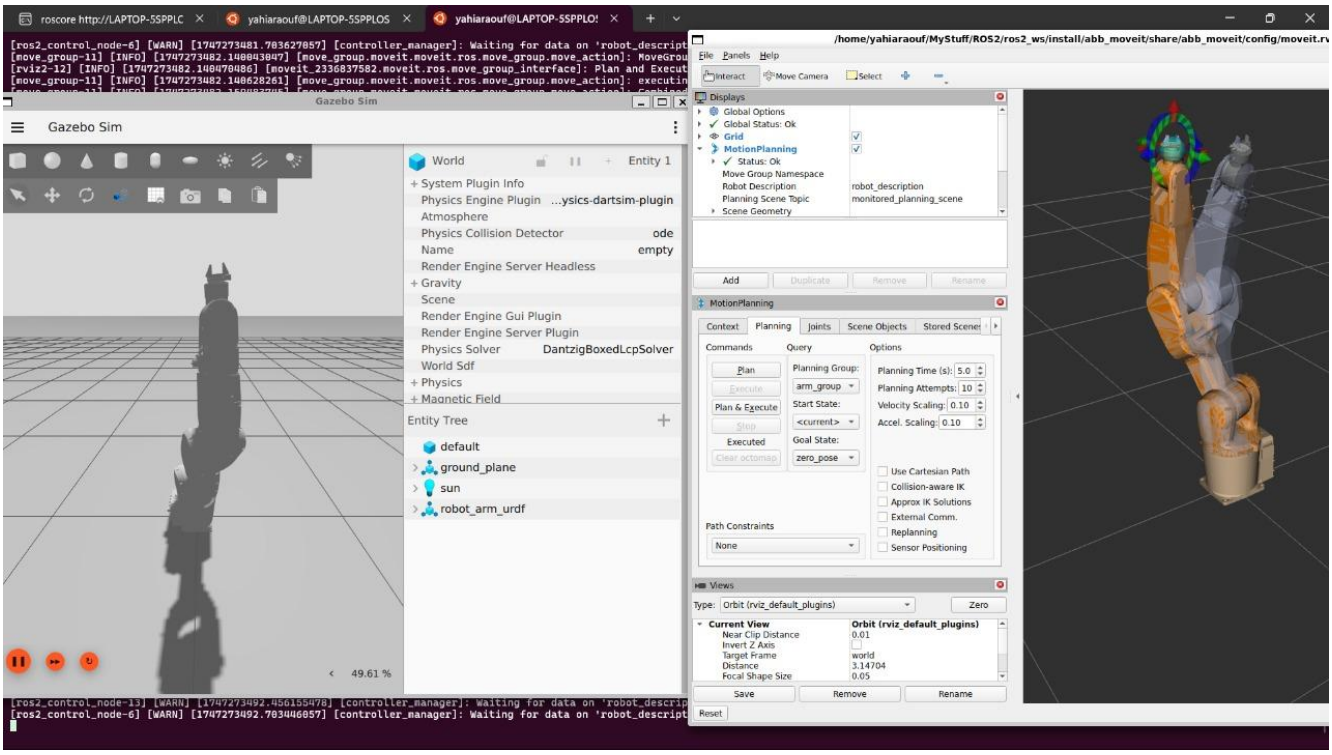

Figure 21:Pose 0

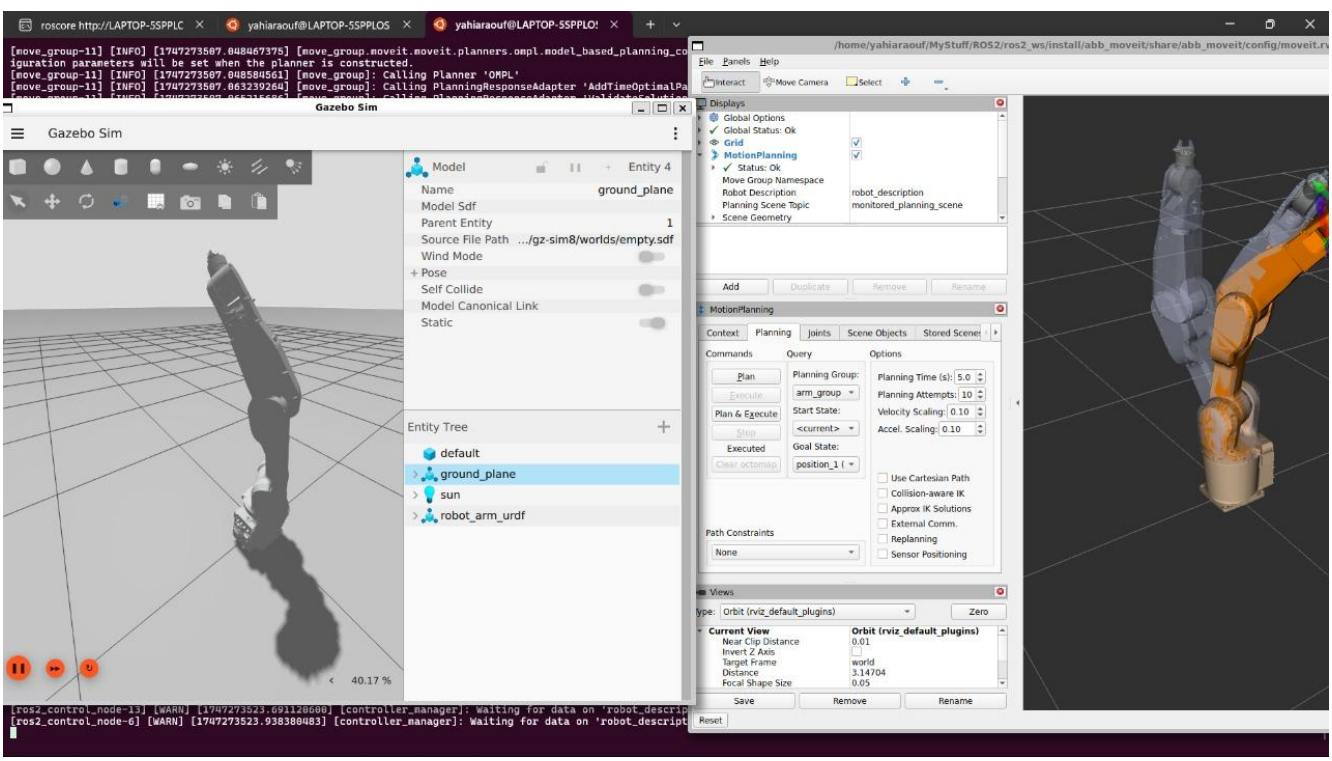

Figure 22:Pose 1

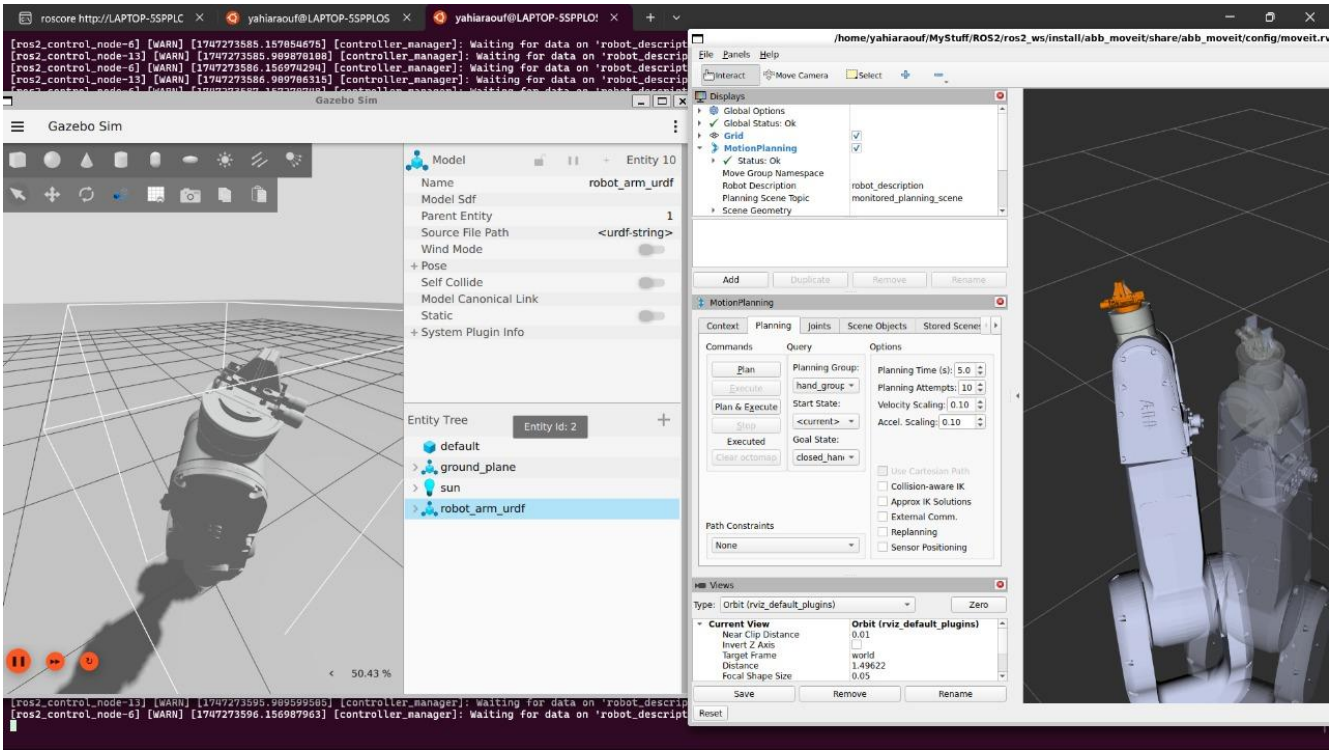

Figure 23:Close Gripper

10.0 Drive Link

https://drive.google.com/drive/folders/1BughekHh_7Fnn9StT0-MOpMM9sKAQMnV?usp=sharing

11.0 Repo Link

https://github.com/YahiaRaouf/Industrial-Robotics-Project.git